



UNIVERSIDADE ESTADUAL DE CAMPINAS
Faculdade de Tecnologia

Victor Valentim Bergamasco

**Sistema Web para Gerenciar a Avaliação de
Disciplinas da Faculdade de Tecnologia da
UNICAMP**

Limeira

2026

Victor Valentim Bergamasco

Sistema Web para Gerenciar a Avaliação de Disciplinas da Faculdade de Tecnologia da UNICAMP

Monografia apresentada à Faculdade de
Tecnologia da Universidade Estadual de
Campinas como parte dos requisitos para a
obtenção do título de Bacharel em Sistemas
de Informação.

Orientador: Prof. Dr. Vitor Rafael Coluci

Este trabalho corresponde à versão final da
Monografia defendida por Victor Valentim
Bergamasco e orientada pelo Prof. Dr. Vitor
Rafael Coluci.

Limeira

2026

FOLHA DE APROVAÇÃO

Abaixo se apresentam os membros da comissão julgadora da sessão pública de defesa de monografia para o Título de Bacharel em Sistemas de Informação, a que se submeteu o aluno Victor Valentim Bergamasco, em na Faculdade de Tecnologia – FT/UNICAMP, em Limeira/SP.

Prof. Dr. Vitor Rafael Coluci

Presidente da Comissão Julgadora

Prof. Dr. Ulisses Martins Dias

Universidade Estadual de Campinas

Prof. Dr. Plínio Roberto Souza Vilela

Universidade Estadual de Campinas

Ata da defesa, assinada pelos membros da Comissão Examinadora, encontra-se no SIGA/ Sistema de Fluxo de Dissertação/Tese e na Secretaria de Graduação da Faculdade de Tecnologia.

Agradecimentos

Agradeço primeiramente à minha família, pelo apoio incondicional, pela paciência e pelo incentivo ao longo de toda a minha formação acadêmica e durante a execução deste trabalho.

Ao meu orientador, Prof. Dr. Vitor Rafael Coluci, registro minha sincera gratidão pela orientação atenciosa, pela disponibilidade contínua e pelas contribuições essenciais à definição do escopo, à modelagem do sistema e à redação desta monografia. Suas observações foram determinantes para o resultado aqui apresentado.

Estendo meus agradecimentos aos professores da Faculdade de Tecnologia da UNICAMP, que ao longo do curso forneceram a base teórica e prática necessária para a realização deste projeto, e aos colegas de turma, com quem dividi experiências de aprendizado e amadurecimento profissional. Por fim, agradeço a todos que, direta ou indiretamente, contribuíram para que esta etapa fosse concluída.

Resumo

Este trabalho apresenta o desenvolvimento de um sistema web para a gestão das avaliações de disciplinas da Faculdade de Tecnologia (FT) da Universidade Estadual de Campinas (UNICAMP). Atualmente, as avaliações são realizadas por meio de QR Codes vinculados a formulários externos, e os dados coletados são entregues em formato bruto a um responsável centralizado pela coordenação do processo, que organiza manualmente os resultados de todas as disciplinas e distribui o relatório individual a cada professor — um trabalho repetitivo, suscetível a erros e que dificulta a padronização dos relatórios. O sistema desenvolvido digitaliza e automatiza o ciclo completo: administradores cadastram professores e disciplinas pela interface web; cada disciplina recebe automaticamente um QR Code único e o conjunto de vinte e quatro perguntas oficiais da FT-UNICAMP; os alunos respondem ao formulário anonimamente; e cada professor acessa diretamente, sem intermediários, um relatório consolidado em PDF da própria disciplina, contendo histogramas, percentuais, médias com desvio padrão, comentários abertos e cálculo do tamanho amostral. A solução foi construída com Django e Django REST Framework no back-end, React no front-end e PostgreSQL como banco de dados. A autenticação utiliza tokens JWT e o anonimato dos alunos é garantido estruturalmente pela ausência de qualquer vínculo entre as respostas e a identidade dos respondentes. Espera-se que o sistema elimine a etapa manual de consolidação centralizada e contribua para a melhoria contínua da qualidade do ensino na FT-UNICAMP.

Abstract

This work presents the development of a web system for managing course evaluations at the School of Technology (FT) of the State University of Campinas (UNICAMP). Currently, evaluations are conducted through QR Codes linked to external forms, and the collected data is delivered in raw format to a coordinator responsible for the process, who must manually organize the results of every course and distribute an individual report to each instructor — a repetitive, error-prone task that hinders the standardization of reports. The developed system digitalizes and automates the entire cycle: administrators register professors and courses through the web interface; each course automatically receives a unique QR Code and a set of twenty-four official questions from FT-UNICAMP; students respond to the form anonymously; and each professor directly accesses a consolidated PDF report of their own course, with no intermediaries, containing histograms, percentages, means with standard deviations, open comments, and sample-size calculations. The solution was built with Django and Django REST Framework on the back-end, React on the front-end, and PostgreSQL as the database. Authentication is implemented through JWT tokens and the anonymity of students is structurally guaranteed by the absence of any link between responses and the identity of respondents. The system is expected to eliminate the manual centralized consolidation step and contribute to the continuous improvement of teaching quality at FT-UNICAMP.

LISTA DE FIGURAS

Figura 1 Diagrama de casos de uso do sistema.	17
Figura 2 Diagrama de atividades do fluxo de autenticação com tratamento de primeiro login.	18
Figura 3 Diagrama de atividades da recuperação de senha por e-mail.	20
Figura 4 Diagrama de atividades do cadastro de disciplina e suas automações.	22
Figura 5 Diagrama de atividades do ciclo completo de avaliação.	24
Figura 6 Modelo Entidade-Relacionamento. A EvaluationResponse não possui referência a User: é a garantia estrutural do anonimato.	25
Figura 7 Fluxo simplificado de duas requisições consecutivas: login seguido de consulta autenticada.	27

LISTA DE QUADROS

Quadro 1	Atores do sistema e suas responsabilidades.	14
Quadro 2	Requisitos funcionais do sistema.	15
Quadro 3	Requisitos não funcionais do sistema.	16
Quadro 4	Matriz de cenários de teste manual.	36

Sumário

1.	Introdução	11
1.1.	Contextualização	11
1.2.	Objetivos	12
1.2.1.	Objetivo geral	12
1.2.2.	Objetivos específicos	12
1.3.	Organização da monografia	12
2.	Levantamento e Análise de Requisitos	13
2.1.	Análise do processo atual	13
2.2.	Atores do sistema	14
2.3.	Requisitos funcionais	14
2.4.	Requisitos não funcionais	15
3.	Modelagem do Sistema	16
3.1.	Diagrama de casos de uso	16
3.2.	Diagramas de atividades	17
3.3.	Modelo de dados	25
4.	Tecnologias e Arquitetura	26
4.1.	Visão geral em três camadas	26
4.2.	Back-end: Django e Django REST Framework	27
4.3.	Front-end: React e Vite	28
4.4.	Banco de dados: PostgreSQL	28
4.5.	Autenticação: JSON Web Tokens	29
4.6.	Geração de relatórios em PDF	29
4.7.	Geração de QR Code	30
5.	Implementação	30
5.1.	Estrutura do projeto	30
5.2.	Aplicação users: autenticação, papéis e primeiro login	31
5.3.	Aplicação courses: QR Code e padronização automáticos	31
5.4.	Aplicação evaluations: perguntas-padrão, formulário público e respostas anônimas	32
5.5.	Fluxos de primeiro login e recuperação de senha	32
5.6.	Geração do relatório PDF	33
6.	Segurança, Anonimato e Privacidade	34
6.1.	Camadas de autenticação e controle de acesso	34
6.2.	Anonimato estrutural dos respondentes	35
6.3.	Conformidade com a LGPD	35
7.	Testes e Validação	36
7.1.	Estratégia de testes	36

7.2.	Cenários testados e resultados	37
7.3.	Limitações da estratégia adotada	38
8.	Hospedagem e Disponibilização	38
8.1.	Execução local	38
8.2.	Caminho para produção	38
9.	Limitações e Trabalhos Futuros	39
9.1.	Limitações atuais	39
9.2.	Trabalhos futuros	40
10.	Conclusão	40
11.	Referências	41
12.	Anexos e Apêndices	42
12.1.	Apêndice A — Links do Projeto	42
12.2.	Apêndice B — Listagens de Código	43
12.2.1.	B.1 Configuração do projeto	43
12.2.2.	B.2 Aplicação users: modelo e permissões	44
12.2.3.	B.3 Aplicação courses: QR Code e padronização automáticos no save()	45
12.2.4.	B.4 Aplicação evaluations: modelos e sincronização	46
12.2.5.	B.5 Fluxos de primeiro login e recuperação de senha	48
12.2.6.	B.6 Geração do relatório PDF	51
12.2.7.	B.7 Filtragem por papel e dashboard	53
12.2.8.	B.8 Front-end: cliente HTTP, Layout e Login	54

1. Introdução

1.1. Contextualização

A avaliação periódica das disciplinas pelos alunos é um instrumento central da gestão da qualidade do ensino superior. Por meio dessas avaliações, instituições e docentes obtêm retorno estruturado sobre infraestrutura, conteúdo programático, métodos didáticos e atuação docente, gerando insumos para a melhoria contínua dos cursos. Esse processo é particularmente relevante em faculdades de tecnologia, nas quais a evolução acelerada das ferramentas e linguagens exige que disciplinas e práticas pedagógicas sejam revistas com frequência.

Na Faculdade de Tecnologia (FT) da Universidade Estadual de Campinas (UNICAMP), a avaliação institucional das disciplinas é realizada ao final de cada semestre por meio de um instrumento composto por vinte e duas perguntas de escala — agrupadas em três blocos (infraestrutura e suporte às aulas, participação do estudante e atuação docente) — e duas questões abertas, totalizando vinte e quatro itens. O instrumento é aplicado de forma digital: os professores compartilham com suas turmas um QR Code que aponta para um formulário externo, e os alunos respondem pelos próprios dispositivos.

Embora a coleta digital represente um avanço em relação ao preenchimento manual, o ciclo atual apresenta uma limitação importante. As respostas são entregues em formato bruto, em planilhas com uma linha por aluno e uma coluna por questão, a uma pessoa responsável pela coordenação do processo — papel hoje exercido pelo próprio orientador deste trabalho. Para extrair valor desse material, essa pessoa precisa, manualmente, separar os dados de cada disciplina, calcular médias, comparar resultados entre questões, produzir um relatório individual para cada professor e, em seguida, distribuir esses relatórios aos docentes — atividade trabalhosa, suscetível a erros, dependente de uma única pessoa e que desestimula análises mais cuidadosas. A consequência é que parte expressiva do potencial do instrumento de avaliação acaba não sendo aproveitada na prática.

Diante dessa lacuna, este trabalho propõe o desenvolvimento de um sistema web que automatiza o ciclo completo da avaliação de disciplinas da FT-UNICAMP. A solução cobre desde o cadastro de professores e disciplinas pelo administrador, com geração automática de QR Code único por disciplina e aplicação do conjunto oficial das vinte e quatro perguntas, até a coleta anônima das respostas dos alunos e a entrega, aos professores, de um relatório consolidado em PDF com histogramas, percentuais, médias com desvio padrão e o conjunto de comentários abertos recebidos.

1.2. Objetivos

1.2.1. Objetivo geral

Desenvolver um sistema web para apoiar a coleta, o armazenamento e a análise das avaliações realizadas pelos alunos da Faculdade de Tecnologia da UNICAMP sobre as disciplinas e seus professores, promovendo uma cultura de feedback contínuo e qualificado e eliminando a etapa manual e centralizada de consolidação e distribuição dos relatórios.

1.2.2. Objetivos específicos

Os objetivos específicos do trabalho são:

1. implementar um sistema com acesso restrito a administradores e professores, autenticados por e-mail institucional e senha;
2. permitir o cadastro, edição e exclusão de professores e disciplinas pelo administrador, com geração automática de QR Codes únicos para cada disciplina;
3. disponibilizar um formulário público de avaliação, acessível por QR Code e respondido anonimamente;
4. armazenar respostas de forma estruturada em banco de dados relacional, sem qualquer vínculo com a identidade do respondente;
5. gerar automaticamente relatórios consolidados em PDF, contendo histogramas, percentuais, médias, desvio padrão, comentários abertos e cálculo do tamanho amostral;
6. garantir que cada professor visualize apenas os dados das próprias disciplinas, preservando a confidencialidade do material avaliativo;
7. implementar fluxos de segurança complementares, incluindo troca obrigatória de senha no primeiro acesso e recuperação de senha por e-mail com link expirável.

1.3. Organização da monografia

O restante deste documento está organizado da seguinte forma. O Capítulo 2 descreve o processo atual de avaliação na FT-UNICAMP, identifica os atores envolvidos e formaliza os requisitos funcionais e não funcionais. O Capítulo 3 apresenta a modelagem do sistema por meio de diagramas UML — casos de uso e atividades — e do modelo de dados relacional. O Capítulo 4 discute as tecnologias adotadas e a arquitetura em três camadas, justificando as escolhas a partir da documentação oficial das ferramentas. O Capítulo 5 descreve a implementação propriamente dita, organizada por aplicações do back-end e por componentes do front-end. O Capítulo 6 trata dos mecanismos de segurança, anonimato e privacidade. O Capítulo 7 apresenta a estratégia de testes adotada. O

Capítulo 8 discute a estratégia de hospedagem. O Capítulo 9 enumera as limitações do trabalho e aponta trabalhos futuros. O Capítulo 10 conclui a monografia. Em seguida, são apresentadas as referências bibliográficas e os apêndices, que incluem o link do repositório do código-fonte e o vídeo demonstrativo do sistema.

2. Levantamento e Análise de Requisitos

Este capítulo descreve, em primeiro lugar, o contexto e o processo atual de avaliação de disciplinas na FT-UNICAMP, identificando os atores envolvidos e as principais limitações que motivaram este trabalho. A partir desse diagnóstico, são formalizados os requisitos funcionais (RFs) e não funcionais (RNFs) que orientaram o projeto e a implementação do sistema.

2.1. Análise do processo atual

O processo atual de avaliação de disciplinas na FT-UNICAMP segue um fluxo simples, porém pouco automatizado. Ao final de cada semestre letivo, a coordenação disponibiliza aos professores um QR Code padronizado, vinculado a um formulário externo, contendo as vinte e quatro perguntas oficiais do instrumento institucional. O professor projeta o QR Code para os alunos durante a aula, e cada aluno acessa o formulário no próprio dispositivo móvel, respondendo de forma anônima.

Após a coleta, os dados consolidados de todas as disciplinas são extraídos em um único arquivo, geralmente em formato de planilha eletrônica, e entregues a uma pessoa responsável pela coordenação do processo — papel hoje exercido pelo próprio orientador deste trabalho. Daí em diante, todo o trabalho analítico é manual e centralizado nessa pessoa: ela precisa, para cada disciplina, separar a fatia correspondente das respostas, calcular as médias por questão, contabilizar as frequências de cada opção (concordo totalmente, concordo parcialmente, neutro, discordo parcialmente, discordo totalmente, não sei avaliar), produzir gráficos, organizar os comentários abertos e formatar um relatório individual que possa ser entregue ao docente responsável. Ao fim do ciclo, esses relatórios são distribuídos um a um aos respectivos professores, que os utilizam em coordenações de curso ou em processos de progressão funcional.

Foi a partir da análise desse processo, somada às conversas com o orientador deste trabalho sobre a rotina dessa atividade, que se identificaram três limitações principais: a primeira, o tempo despendido pela pessoa responsável em uma tarefa repetitiva e padronizada, que escala com o número de disciplinas avaliadas; a segunda, a centralização de todo o ciclo em uma única pessoa, que se torna gargalo para que os relatórios cheguem

aos docentes; e a terceira, o risco de erros de tabulação ou de distribuição, especialmente quando o número de respondentes é elevado. Tais limitações justificam a substituição da etapa manual por um sistema que entregue os relatórios já consolidados e padronizados diretamente a cada professor, sem intermediários.

2.2. Atores do sistema

A partir da análise do processo atual, foram identificados três atores principais que interagem com o sistema desenvolvido. O Quadro 1 sintetiza as responsabilidades e o nível de acesso de cada um.

Ator	Responsabilidades
Administrador	Cadastra e gerencia professores e disciplinas; edita o template de perguntas-padrão; acompanha estatísticas globais. Acesso total ao sistema.
Professor	Visualiza suas próprias disciplinas; exibe QR Code aos alunos; consulta painel com médias por pergunta; baixa relatórios em PDF. Acesso restrito às próprias disciplinas.
Aluno	Respondente anônimo do formulário. Acessa o instrumento exclusivamente pelo QR Code da disciplina; não realiza login.

Quadro 1: Atores do sistema e suas responsabilidades.

A separação clara de papéis fundamenta todas as decisões de controle de acesso descritas no Capítulo 6 e a estrutura de modelagem apresentada no Capítulo 3.

2.3. Requisitos funcionais

Os requisitos funcionais foram derivados diretamente da análise do processo atual e das conversas mantidas com o orientador deste trabalho. Cada requisito foi formulado de modo a ser verificável — isto é, expresso como uma capacidade observável do sistema, sujeita a teste manual ou automatizado. O Quadro 2 apresenta os dez requisitos funcionais identificados.

ID	Requisito
RF01	O sistema deve permitir o login de administradores e professores mediante autenticação por e-mail institucional e senha.
RF02	O administrador deve poder cadastrar, editar e excluir professores no sistema.
RF03	O administrador deve poder cadastrar, editar e excluir disciplinas, associando cada disciplina a um professor responsável e informando o número de matriculados.
RF04	O sistema deve gerar automaticamente um QR Code único por disciplina, apontando para o formulário público correspondente.
RF05	O sistema deve apresentar ao administrador uma tela inicial com estatísticas globais (número de professores, disciplinas e respostas).
RF06	O professor deve visualizar somente as disciplinas pelas quais é responsável, com acesso ao QR Code de cada uma.
RF07	O sistema deve permitir que professores e administradores baixem relatórios consolidados em PDF, contendo histogramas, médias, desvios padrão e comentários por disciplina.
RF08	O sistema deve impedir que um professor acesse relatórios de disciplinas pelas quais não é responsável.
RF09	O sistema deve oferecer um formulário público de avaliação, acessível por QR Code e sem necessidade de autenticação do aluno.
RF10	O sistema deve oferecer ao usuário um fluxo de recuperação de senha por e-mail, com link de redefinição válido por tempo limitado.

Quadro 2: Requisitos funcionais do sistema.

2.4. Requisitos não funcionais

Os requisitos não funcionais expressam restrições e qualidades do sistema, e foram derivados das mesmas conversas e da análise do contexto. Os RNFs foram redigidos de forma atômica — cada item trata de uma única restrição — para facilitar a sua verificação. O Quadro 3 apresenta os requisitos não funcionais adotados.

ID	Requisito
RNF01	O sistema deve ser desenvolvido com React no front-end, Django e Django REST Framework no back-end e PostgreSQL como banco de dados.
RNF02	A comunicação entre front-end e back-end deve ocorrer exclusivamente por meio de API RESTful, com mensagens em formato JSON.
RNF03	A autenticação dos usuários deve ser baseada em tokens JWT (JSON Web Tokens).
RNF04	A interface deve ser responsiva, adaptando-se a navegadores em computadores e dispositivos móveis.
RNF05	O sistema deve garantir o anonimato das respostas, não armazenando qualquer vínculo entre a identidade do aluno e os dados submetidos no formulário.
RNF06	As senhas dos usuários devem ser armazenadas em forma de hash, com algoritmo seguro recomendado pela documentação do Django.
RNF07	O sistema deve manter um conjunto de vinte e quatro perguntas-padrão centralizadas, replicáveis automaticamente em toda disciplina nova.
RNF08	A geração de relatórios em PDF deve ocorrer no servidor, de forma a garantir consistência visual independentemente do dispositivo do usuário.

Quadro 3: Requisitos não funcionais do sistema.

3. Modelagem do Sistema

Este capítulo apresenta a modelagem do sistema, servindo como elo entre os requisitos discutidos no Capítulo 2 e a implementação descrita no Capítulo 5. São apresentados, em ordem, o diagrama de casos de uso, os diagramas de atividades dos principais fluxos operacionais e o modelo conceitual de dados, que orienta a estrutura das tabelas no banco relacional.

3.1. Diagrama de casos de uso

O diagrama de casos de uso da *Unified Modeling Language* (UML) representa, de modo gráfico, as funcionalidades do sistema do ponto de vista de seus atores. No sistema desenvolvido, o ator Administrador realiza casos de uso voltados ao gerenciamento de cadastros — manutenção de professores, manutenção de disciplinas, edição das perguntas-padrão e consulta da visão geral — além dos casos compartilhados de autenticação e troca de senha. O ator Professor compartilha os casos de autenticação e visualiza as próprias disciplinas, consulta o QR Code (após a geração automática feita pelo sistema), visualiza o painel agregado e baixa o relatório PDF. O ator Aluno, não autenticado,

executa unicamente o caso de uso responder formulário, acessado por meio do QR Code da disciplina.

A Figura 1 ilustra o relacionamento entre atores e casos de uso, organizando os subsistemas que serão detalhados nos capítulos seguintes.

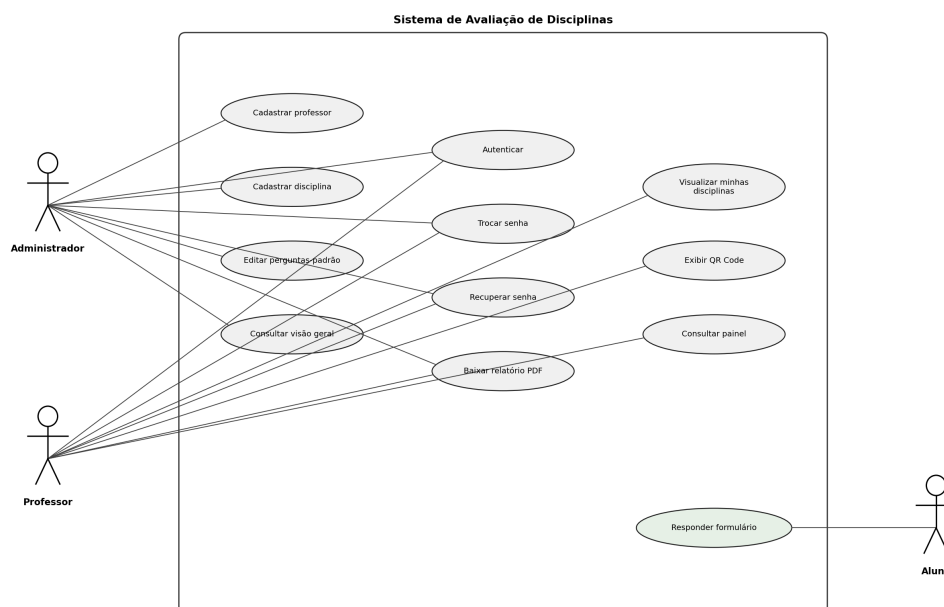


Figura 1: Diagrama de casos de uso do sistema.

Fonte: Elaborado pelo autor.

3.2. Diagramas de atividades

Os diagramas de atividades descrevem, de forma operacional, a sequência de passos executados em cada um dos principais fluxos do sistema. Quatro fluxos foram modelados em detalhe: autenticação com tratamento de primeiro login (Figura 2); recuperação de senha por e-mail (Figura 3); cadastro de uma nova disciplina, com geração automática de QR Code e replicação das perguntas-padrão (Figura 4); e ciclo completo de avaliação, desde o escaneamento do QR Code pelo aluno até a geração do relatório PDF pelo professor (Figura 5).

A Figura 2 descreve o fluxo de autenticação. O usuário informa e-mail e senha; o sistema valida e gera um token JWT. Em seguida, o front-end consulta o endpoint que retorna os dados do usuário corrente, com o propósito de obter o papel e o estado da flag de primeiro acesso. Se o usuário precisa trocar a senha (caso típico do primeiro login), o sistema bloqueia o acesso a qualquer outra tela e redireciona para o fluxo de definição de senha. Caso contrário, o usuário é enviado ao painel apropriado.

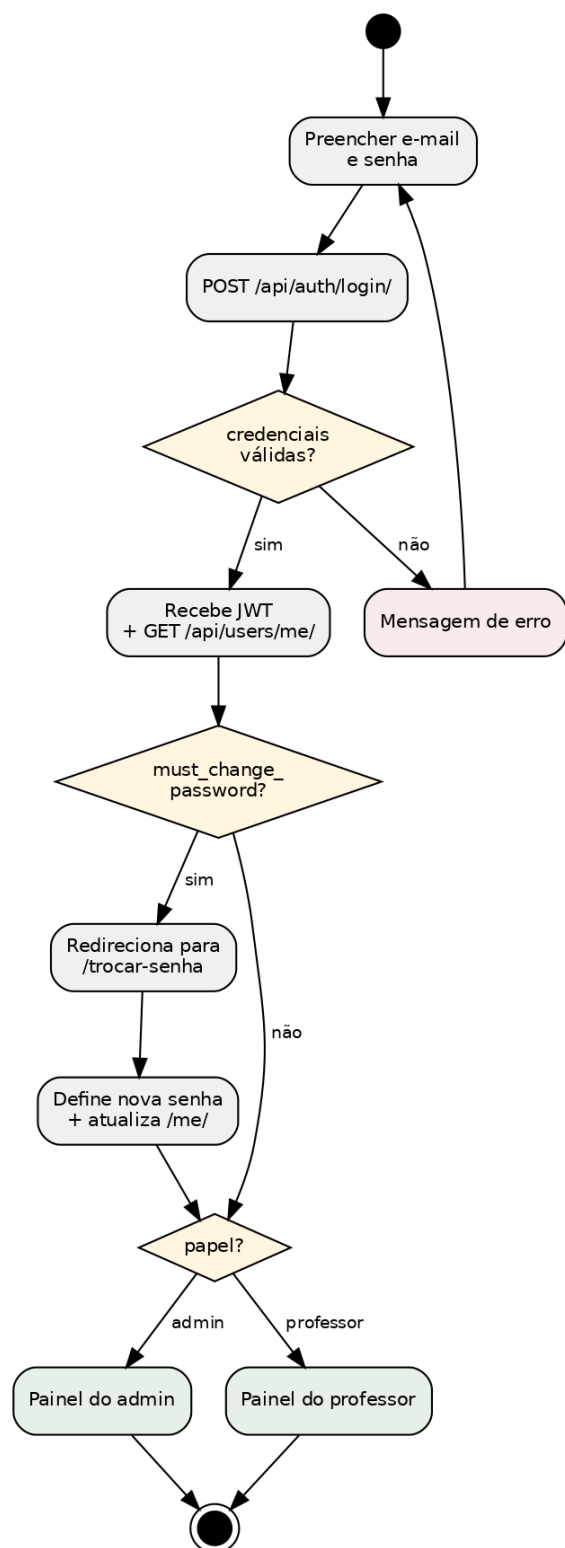


Figura 2: Diagrama de atividades do fluxo de autenticação com tratamento de primeiro login.

Fonte: Elaborado pelo autor.

A Figura 3 descreve o fluxo de recuperação de senha. O usuário solicita a recuperação informando seu e-mail; o sistema verifica se existe um usuário ativo com aquele e-mail e, em caso afirmativo, gera um token vinculado ao *hash* atual da senha e envia o *link* por e-mail. A resposta ao *front-end* é sempre genérica, independentemente de o e-mail existir, evitando vazamento de informação sobre quais e-mails estão cadastrados.

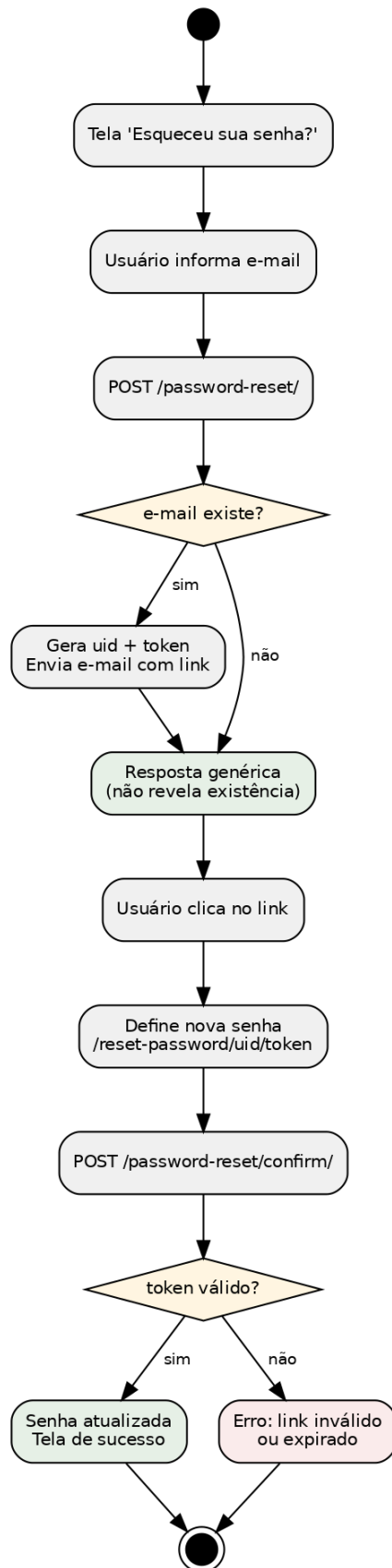


Figura 3: Diagrama de atividades da recuperação de senha por e-mail.

Fonte: Elaborado pelo autor.

A Figura 4 descreve o fluxo de cadastro de disciplina, com ênfase em duas automações que ocorrem no momento da criação: a geração do QR Code único (a partir do link do formulário público) e a replicação do template de perguntas-padrão, criando, para a nova disciplina, cópias de todas as vinte e quatro perguntas oficiais.

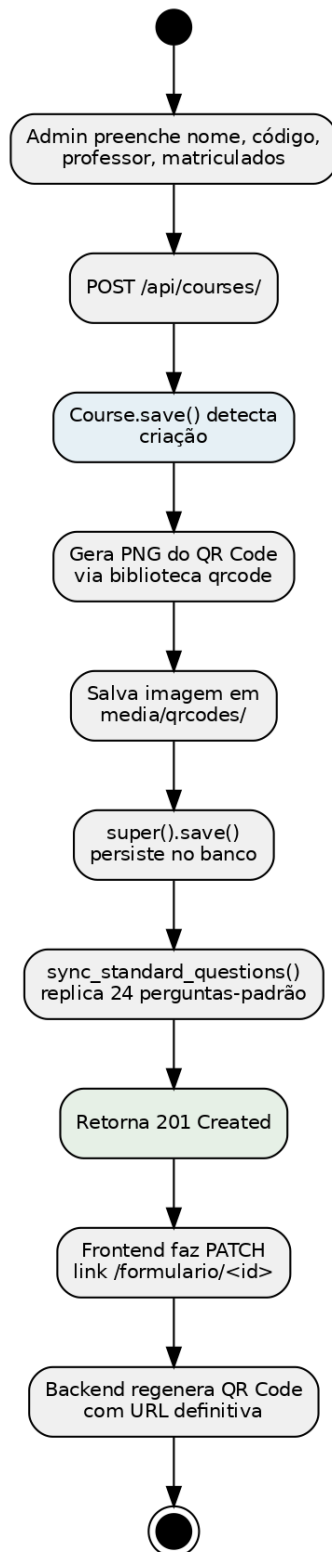


Figura 4: Diagrama de atividades do cadastro de disciplina e suas automações.

Fonte: Elaborado pelo autor.

A Figura 5 apresenta o fluxo completo de avaliação, do escaneamento do QR Code pelo aluno até a geração do relatório PDF pelo professor. O diagrama deixa explícito o ponto

em que a separação entre dados do respondente e do sistema ocorre: o formulário público é acessível sem autenticação e a resposta é armazenada sem qualquer vínculo com a identidade do aluno.

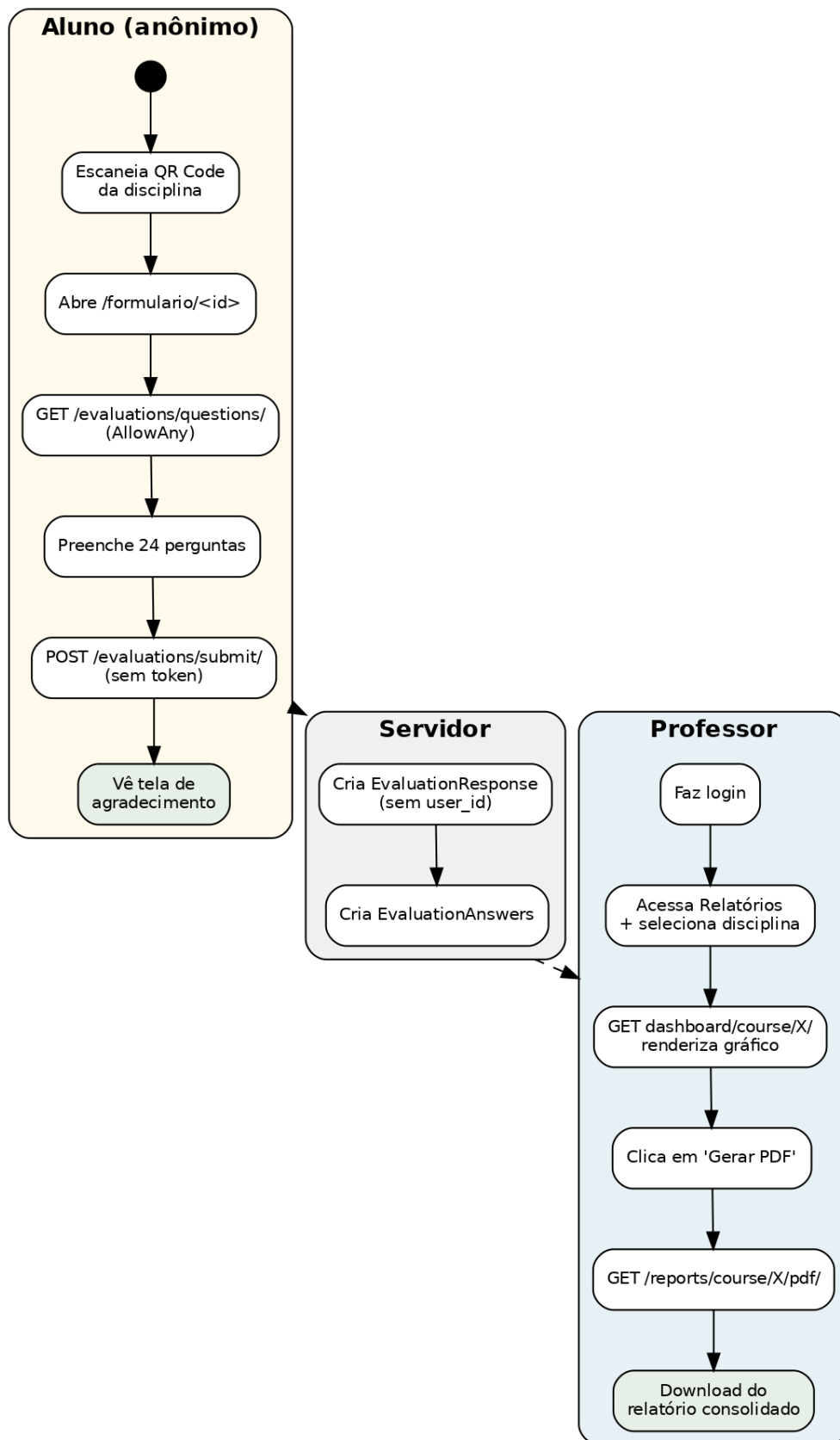


Figura 5: Diagrama de atividades do ciclo completo de avaliação.

Fonte: Elaborado pelo autor.

3.3. Modelo de dados

O modelo conceitual de dados do sistema é composto por cinco entidades principais, relacionadas por meio de chaves estrangeiras com regra de exclusão em cascata. A entidade *User* representa administradores e professores, com login por e-mail institucional, papel (admin ou professor) e o atributo de primeiro login (*must_change_password*), que força a redefinição de senha no primeiro acesso. A entidade *Course* representa as disciplinas: tem nome, código único, professor responsável (referência a *User*), número de matriculados, link do formulário público e referência para a imagem do QR Code gerado automaticamente. A entidade *StandardQuestion* atua como template das perguntas-padrão da FT-UNICAMP — ordem, texto, tipo (escala ou texto livre) e ativação — e é replicada para cada nova disciplina como entidades *EvaluationQuestion*, que pertencem a uma *Course* específica.

As respostas dos alunos são representadas por duas entidades intencionalmente desvinculadas de *User*. A entidade *EvaluationResponse* registra uma submissão completa do formulário, com data e hora e referência apenas à *Course* — jamais ao respondente. A entidade *EvaluationAnswer* detalha cada resposta individual a uma *EvaluationQuestion*, armazenando um valor de escala (zero a cinco) para perguntas quantitativas ou um valor textual para perguntas abertas. A ausência deliberada de qualquer ligação entre *EvaluationResponse* e *User* é a garantia estrutural do anonimato exigido pelo instrumento de avaliação institucional. A Figura 6 apresenta o esqueleto do modelo entidade-relacionamento (DER) implementado.

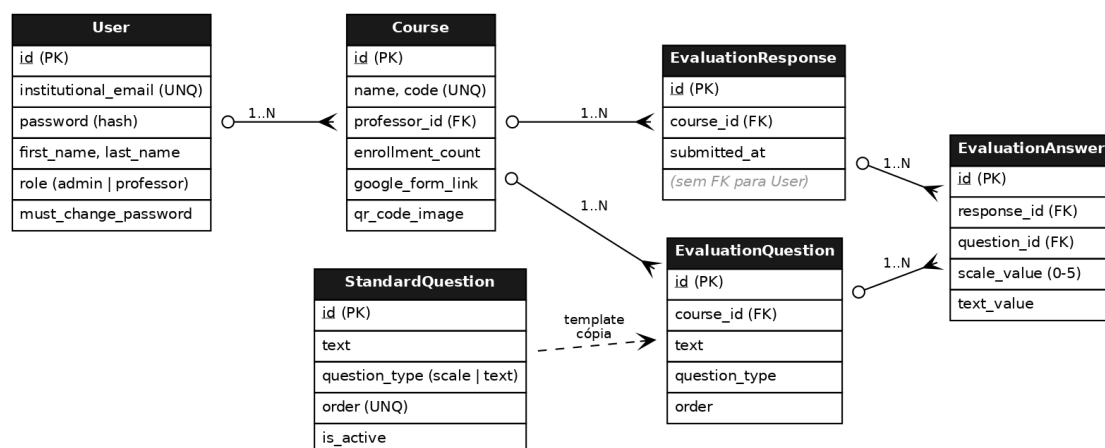


Figura 6: Modelo Entidade-Relacionamento. A *EvaluationResponse* não possui referência a *User*: é a garantia estrutural do anonimato.

Fonte: Elaborado pelo autor.

4. Tecnologias e Arquitetura

Este capítulo apresenta as tecnologias adotadas no desenvolvimento do sistema e a arquitetura que organiza a sua implementação. A discussão parte de uma visão geral da arquitetura em três camadas, justifica a escolha do estilo arquitetural cliente-servidor com API REST e, em seguida, detalha cada tecnologia, indicando a versão adotada e citando a documentação oficial das ferramentas.

4.1. Visão geral em três camadas

O sistema segue o padrão cliente-servidor clássico em três camadas. A camada de apresentação executa no navegador do usuário e é responsável pela interação direta — exibição de telas, captura de entrada e renderização de gráficos. A camada de aplicação executa no servidor e centraliza a lógica de negócio, validações, controle de acesso, geração de relatórios em PDF e geração de QR Codes. A camada de dados, também no lado do servidor, armazena de forma persistente as informações em um banco de dados relacional.

A comunicação entre as camadas de apresentação e de aplicação ocorre exclusivamente por meio de uma API HTTP no estilo REST, com mensagens trocadas em formato JSON. Essa separação estrita — imposta tanto pelo requisito RNF02 quanto pelo desacoplamento entre front-end e back-end — permite que cada camada evolua de modo independente, e abre caminho para futuras extensões, como um eventual aplicativo móvel que reutilize a mesma API. A Figura 7 representa esquematicamente o fluxo de uma requisição típica.

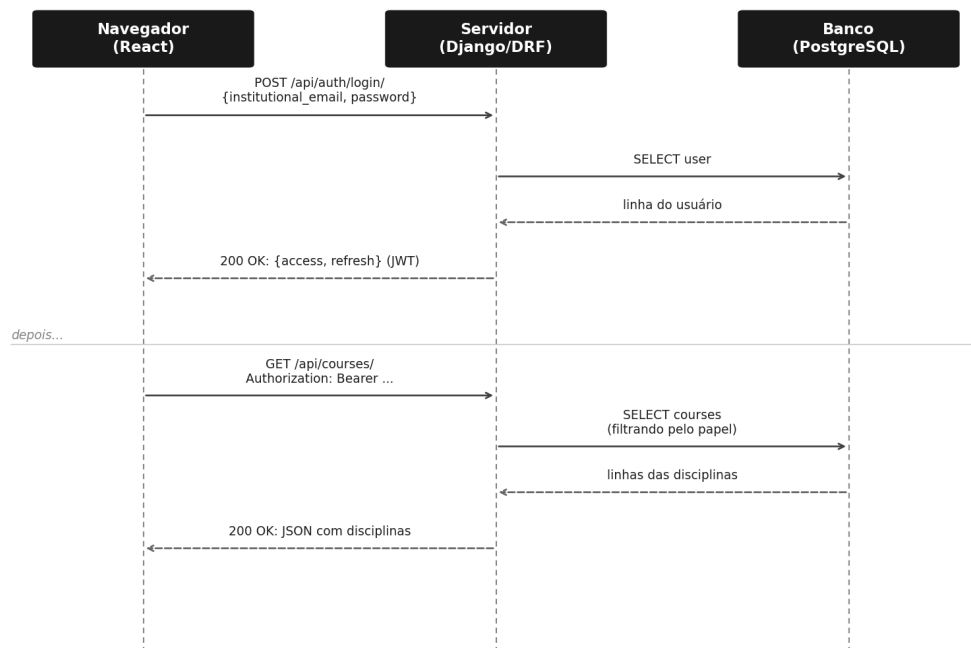


Figura 7: Fluxo simplificado de duas requisições consecutivas: login seguido de consulta autenticada.

Fonte: Elaborado pelo autor.

4.2. Back-end: Django e Django REST Framework

A camada de aplicação foi implementada em Python (versão 3.11), utilizando o framework Django (versão 5.0) e a biblioteca Django REST Framework (DRF, versão 3.15). O Django é descrito por seus mantenedores como um framework web Python de alto nível que incentiva o desenvolvimento rápido e o design limpo e pragmático (Django Software Foundation, 2024). Suas principais contribuições para este projeto foram três: um *Object-Relational Mapper* (ORM) maduro, que abstrai consultas SQL e oferece migrações versionadas; um sistema declarativo de URLs; e validadores de senha integrados, que aplicam regras de comprimento mínimo e complexidade.

Sobre essa base, o Django REST Framework oferece serializadores expressivos, vistas baseadas em classes especializadas em APIs REST e um sistema declarativo de permissões (Django REST Framework, 2024). O DRF foi adotado para reduzir a quantidade de código de transformação entre objetos do banco e respostas JSON, e para permitir a definição concisa de regras de acesso por *endpoint*. A combinação Django + DRF mostrou-se especialmente produtiva no contexto deste TCC, em que o objetivo principal

era a entrega de funcionalidades completas em prazo limitado, sem reinventar componentes já consolidados da comunidade.

4.3. Front-end: React e Vite

A camada de apresentação foi construída em React (versão 18), uma biblioteca JavaScript para construção de interfaces de usuário baseada em componentes reativos (Meta Platforms, 2024). O modelo de componentes do React favorece o reuso e a separação de responsabilidades: cada tela do sistema é uma composição de componentes menores — *cards*, tabelas, formulários — que encapsulam estado e comportamento. O empacotamento e o servidor de desenvolvimento são fornecidos pelo Vite, descrito por seus mantenedores como uma ferramenta de *build* de próxima geração para o desenvolvimento web moderno (You, 2024), responsável pelo *hot module replacement* e pelo *build* final de produção.

Três bibliotecas auxiliares completam o front-end. O React Router gerencia a navegação entre telas em uma aplicação de página única, aplicando regras de proteção de rotas conforme a autenticação do usuário. O Axios é utilizado como cliente HTTP, abstraindo o consumo da API e permitindo a injeção automática do token JWT em cada requisição. A biblioteca Recharts é empregada para a renderização do gráfico de barras no painel de relatórios.

4.4. Banco de dados: PostgreSQL

A camada de dados utiliza o PostgreSQL (versão 14 ou superior), um sistema gerenciador de banco de dados relacional de código aberto descrito como o sistema de banco de dados relacional de código aberto mais avançado do mundo pela sua própria comunidade (PostgreSQL Global Development Group, 2024). O PostgreSQL foi escolhido por três motivos: a integridade referencial nativa via chaves estrangeiras com cascata, indispensável ao modelo de dados apresentado na Seção 3.3; a maturidade do suporte a transações ACID, importante para a consistência da submissão do formulário, que insere um registro de resposta e várias respostas individuais em conjunto; e a integração direta com o ORM do Django, que dispensa a escrita manual de SQL na camada de aplicação.

Em ambiente local, o banco roda na própria máquina do desenvolvedor; suas credenciais são lidas pelo Django a partir de variáveis declaradas em um arquivo `.env`, garantindo que o código-fonte não contenha segredos em texto claro. Em produção, basta apontar as mesmas variáveis para a instância gerenciada do provedor de hospedagem, sem necessidade de alterar nenhuma linha de código.

4.5. Autenticação: JSON Web Tokens

A autenticação do sistema utiliza JSON Web Tokens (JWT), conforme a RFC 7519, que define um meio compacto e auto-contido para a transmissão segura de informações entre partes (Jones; Bradley; Sakimura, 2015). Cada token é composto por três segmentos — cabeçalho, *payload* e assinatura — codificados em Base64URL e separados por pontos; o segmento de assinatura permite ao servidor verificar a autenticidade do token sem manter estado de sessão. No projeto, foi adotada a biblioteca `django-rest-framework-simplejwt`, que integra a emissão e a validação de tokens diretamente às vistas do DRF (Simple JWT, 2024). Tokens de acesso expiram em quatro horas; tokens de *refresh*, em sete dias.

Os tempos de expiração foram dimensionados pensando em uma jornada típica de uso do sistema: quatro horas cobrem uma manhã ou tarde de trabalho do professor, e sete dias do *refresh* permitem que o usuário permaneça conectado por uma semana sem precisar refazer *login*. Para reforçar a segurança, o token é armazenado pelo front-end apenas em *localStorage* e é descartado no *logout*, e a senha do usuário é validada em servidor com *hash* PBKDF2 (configuração padrão do Django).

4.6. Geração de relatórios em PDF

A produção dos relatórios consolidados ocorre integralmente no servidor, conforme o requisito RNF08, com o uso combinado de duas bibliotecas Python. A biblioteca Matplotlib (Hunter, 2007) é empregada na geração dos histogramas de cada pergunta de escala: para cada questão, são calculadas as frequências absolutas e relativas das seis opções da escala (incluindo “Não sei avaliar”), e um gráfico de barras com porcentagens sobrepostas é renderizado e salvo como imagem temporária. A biblioteca ReportLab (ReportLab, 2024) é então utilizada para compor o PDF final em formato A4: cabeçalho institucional, tabela com dados da disciplina, cálculo amostral, seções A, B, C e D do instrumento, histogramas com legendas e comentários abertos numerados.

A geração em servidor garante consistência visual entre relatórios, independentemente do navegador ou do sistema operacional do professor, e isola a lógica de cálculo estatístico (média e desvio padrão) em um módulo único, reutilizável e testável. O cálculo do tamanho amostral utiliza a fórmula clássica para populações finitas, com $z = 1,96$ (95% de confiança), proporção $p = 0,5$ e margem de erro $e = 0,15$ para relatórios individuais por professor. As respostas “Não sei avaliar” são desconsideradas no cálculo da média de cada pergunta, evitando que respostas neutras viciem o resultado, mas são contadas no histograma para preservar a representação fiel das frequências.

4.7. Geração de QR Code

O QR Code de cada disciplina é gerado no momento do cadastro, no próprio servidor, com o uso da biblioteca Python qrcode (Luth, 2024). A imagem resultante é armazenada no diretório de mídia configurado pelo Django e é servida ao front-end por meio de URL absoluta, permitindo que o navegador do professor (ou de quem mais consulte a tela) renderize a imagem como qualquer outro recurso público. A regeneração ocorre automaticamente sempre que o *link* do formulário público associado à disciplina muda — por exemplo, no momento em que o sistema atualiza o *link* da forma genérica `formulario/0` para `formulario/[id_da_disciplina]` logo após a criação.

5. Implementação

Este capítulo descreve a implementação do sistema, traduzindo os requisitos do Capítulo 2 e a modelagem do Capítulo 3 em estrutura de código concreta. A descrição parte da organização geral do projeto, percorre cada uma das aplicações do back-end, e detalha os fluxos mais sofisticados: padronização das perguntas, primeiro login obrigatório, recuperação de senha por e-mail e geração do relatório PDF. A discussão privilegia a descrição em prosa do comportamento esperado e das decisões de projeto, mantendo o foco sobre os conceitos e não sobre a sintaxe das linguagens utilizadas. O leitor interessado em trechos de código pode consultar o repositório referenciado no Apêndice A.

5.1. Estrutura do projeto

O código-fonte é organizado em duas pastas principais, refletindo a separação entre as camadas de apresentação e de aplicação. A pasta `backend_tcc` contém o projeto Django, com a configuração central em `core/` e cinco aplicações em `apps/`: `users` (autenticação e usuários), `courses` (disciplinas e QR Code), `evaluations` (perguntas e respostas), `dashboard` (estatísticas) e `reports` (geração de PDF). A pasta `frontend_tcc` contém a aplicação React criada com Vite. Cada uma das duas pastas é auto-contida: possui suas próprias dependências (declaradas em `requirements.txt` e `package.json`, respectivamente) e é executável de forma independente, comunicando-se com a outra unicamente por meio da API REST.

Para facilitar a operação local, foi escrito um script chamado `iniciar-sistema.bat`, posicionado na raiz do projeto, que verifica a existência do ambiente virtual Python e do diretório `node_modules`, instala automaticamente as dependências do front-end caso ausentes, e abre as duas janelas de servidor em sequência, finalizando com a abertura do navegador no endereço do front-end. O roteamento global do back-end, definido no

arquivo `core/urls.py`, encadeia os arquivos `urls.py` de cada aplicação sob o prefixo `/api/`, além de expor os endpoints padronizados de autenticação JWT.

5.2. Aplicação users: autenticação, papéis e primeiro login

A aplicação `users` é a mais sensível do sistema. Define o modelo de usuário customizado, autenticado por e-mail institucional em vez do `username` padrão do Django; concentra as lógicas de primeiro login obrigatório e de recuperação de senha. O modelo `User` estende `AbstractUser` do Django, substituindo o campo `username` por `institutional_email` e adicionando dois campos específicos do projeto: `role` (papel) e `must_change_password` (indicador booleano de primeiro acesso). Ao cadastrar um professor, o administrador define uma senha provisória; o sistema marca a *flag* de primeiro login e, no primeiro acesso, força o professor a definir uma nova senha pessoal antes de operar qualquer outra tela do sistema.

As permissões são expressas por duas classes customizadas — `IsAdminRole` e `IsProfessorOrAdmin` — aplicadas declarativamente em cada *endpoint* da API. A primeira exige que o usuário esteja autenticado e tenha papel `admin`; a segunda aceita ambos os papéis. Cada *endpoint* da API declara qual classe utiliza, e o Django REST Framework verifica essa regra antes de qualquer código de *view* executar. Além das permissões, os *endpoints* que retornam dados relacionados a disciplinas aplicam, internamente, um filtro adicional sobre o *queryset*: o professor recebe somente as próprias disciplinas, e qualquer tentativa de acessar dados de outra disciplina (por exemplo, manipulando o identificador na URL) é bloqueada pela combinação permissão + filtro.

5.3. Aplicação courses: QR Code e padronização automáticos

A aplicação `courses` define o modelo `Course` (disciplina) e é responsável por duas automações essenciais do sistema: a geração do QR Code no momento da criação da disciplina e a replicação das perguntas-padrão. Ambas são implementadas por sobrecargas do método `save()` do modelo, garantindo que ocorram sempre que uma disciplina é salva, sem precisar lembrar de chamar funções adicionais a partir do código de *views* ou de serializadores.

O método `save()` sobrecarregado primeiro detecta se a chamada é uma criação ou uma edição, observando o estado interno do objeto. Em seguida, decide se o QR Code precisa ser gerado ou regenerado: a geração ocorre na criação e também quando o *link* do formulário público é alterado, garantindo que o QR Code reflita sempre a URL correta. A geração propriamente dita utiliza a biblioteca `qrcode` em conjunto com o `ImageField`

do Django: a imagem é construída em memória, recebe um nome baseado no código da disciplina e é entregue ao *framework*, que se encarrega da persistência em disco. Por fim, se a chamada foi uma criação, o sistema dispara a função `sync_standard_questions`, que replica para a nova disciplina cada uma das perguntas-padrão ativas.

5.4. Aplicação evaluations: perguntas-padrão, formulário público e respostas anônimas

A aplicação `evaluations` contém quatro modelos centrais. O modelo `StandardQuestion` serve como template global das perguntas oficiais da FT-UNICAMP, com ordem única, texto, tipo (escala ou texto livre) e indicador de ativação. Os modelos `EvaluationQuestion` e `EvaluationResponse` representam, respectivamente, as perguntas específicas de cada disciplina (replicadas do template) e as submissões dos alunos. O modelo `EvaluationAnswer` detalha cada resposta individual a uma `EvaluationQuestion`, com um valor de escala (zero a cinco) para perguntas quantitativas ou um valor textual para perguntas abertas.

Para evitar duplicação de lógica entre front-end e back-end, optou-se por concentrar a fonte da verdade das perguntas-padrão no próprio servidor, em um módulo Python que combina uma lista *hardcoded* (usada como *seed*) e uma função idempotente de sincronização (`sync_standard_questions`). A primeira leitura do banco — após a migração inicial — popula automaticamente a tabela `StandardQuestion` a partir da lista *hardcoded*, dispensando comando manual. A função de sincronização pode ser chamada quantas vezes for necessário sobre o mesmo curso, sem criar duplicatas: cria apenas o que falta. Esse comportamento é fundamental para suportar o botão “Aplicar em todas as disciplinas” da tela administrativa de perguntas-padrão, que dispara o `sync` sobre todos os cursos existentes.

Adicionalmente, o administrador pode criar, editar e remover perguntas-padrão pela interface, em uma tela dedicada. Para preservar disciplinas já cadastradas — cujas `EvaluationQuestion` podem ter respostas vinculadas — o sistema não propaga edições automaticamente para os cursos existentes; oferece, na tela, dois botões: um aplica somente as perguntas faltantes, e outro também sobrescreve o texto e o tipo das perguntas existentes. A propagação é, portanto, sempre uma ação consciente do administrador.

5.5. Fluxos de primeiro login e recuperação de senha

O fluxo de primeiro login ocorre quando um administrador cadastra um novo professor, definindo uma senha provisória e marcando o usuário com a *flag* de primeiro acesso. No

primeiro *login*, o *front-end* consulta o *endpoint* `/api/users/me/`, identifica a *flag* ativa e redireciona o usuário para a tela de definição de senha. O usuário informa apenas a nova senha — sem precisar repetir a provisória — e o sistema, no *endpoint* correspondente, valida o tamanho mínimo, aplica o *hash* com o algoritmo padrão do Django e desmarca a *flag*. A proteção de rotas no *front-end* utiliza um componente chamado `PrivateRoute`, que verifica simultaneamente a presença do token e o estado da *flag*, redirecionando o usuário de volta à tela de troca de senha enquanto ela estiver ativa. Esse comportamento garante que o sistema nunca seja operado com a senha provisória definida pelo administrador.

O fluxo de recuperação de senha começa quando o usuário clica em “Esqueceu sua senha?” na tela de *login*. O *front-end* coleta apenas o e-mail institucional e envia um POST para o *endpoint* de solicitação de *reset*. No servidor, o sistema verifica se existe um usuário ativo com aquele e-mail; em caso positivo, gera um *token* único através do utilitário `PasswordResetTokenGenerator` do Django, vinculado ao *hash* atual da senha (de modo que o *token* é invalidado automaticamente após a troca). Em seguida, monta um *link* no formato `FRONTEND_URL/reset-password/{uid}/{token}` e o envia por e-mail. Independentemente da existência do usuário, a resposta da API é sempre HTTP 200 com uma mensagem genérica, para não revelar quais e-mails estão cadastrados. Em ambiente de desenvolvimento, o *backend* de e-mail do Django está configurado para a saída do console — o *link* de redefinição é impresso no terminal do servidor, o que facilita a depuração sem dependência de SMTP. Para produção, basta substituir `EMAIL_BACKEND` para a versão SMTP e preencher as credenciais correspondentes no arquivo `.env`.

5.6. Geração do relatório PDF

A geração do relatório PDF é implementada na aplicação `reports`, em um módulo `services.py` que orquestra a combinação entre `Matplotlib` (para os histogramas) e `ReportLab` (para a montagem do documento). A função principal recebe um objeto `Course` e produz um arquivo PDF formatado em A4, salvo no diretório de mídia e devolvido ao navegador do professor como anexo. O fluxo interno consiste em três etapas: primeiro, são calculados os dados gerais da disciplina — número de respondentes, número de matriculados, razão entre os dois e tamanho amostral mínimo; em seguida, para cada pergunta de escala, calcula-se a frequência de cada opção (de zero a cinco), gera-se o histograma com porcentagens sobrepostas e calculam-se a média e o desvio padrão das respostas válidas; por fim, para cada pergunta de texto livre, lista-se o conjunto numerado dos comentários submetidos. O `ReportLab` compõe o documento agrupando as perguntas nas quatro seções oficiais (A, B, C e D) e aplicando rodapé personalizado em todas as páginas.

Duas decisões merecem registro. A primeira é a exclusão da opção “Não sei avaliar” do cálculo da média e do desvio padrão: respostas neutras inflariam artificialmente o numerador da média e enviesariam a interpretação do resultado. A segunda é a indicação visual do cálculo amostral no cabeçalho do relatório: quando o número de respondentes é inferior ao mínimo estatístico, o sistema inclui uma observação destacada, alertando o leitor para a baixa significância estatística da amostra.

6. Segurança, Anonimato e Privacidade

Este capítulo descreve os mecanismos adotados para proteger o sistema, garantir o anonimato dos respondentes e respeitar os princípios da Lei Geral de Proteção de Dados Pessoais (LGPD). A discussão parte das camadas técnicas de proteção, passa pela garantia estrutural do anonimato e termina na análise da conformidade legal.

6.1. Camadas de autenticação e controle de acesso

A segurança do sistema apoia-se em cinco camadas complementares. A primeira é a autenticação por JWT, em conformidade com o requisito RNF03: o *token* é gerado no *login* e exigido em cada requisição a *endpoints* protegidos. A segunda camada é o armazenamento das senhas em forma de *hash* (RNF06): o Django utiliza, por padrão, o algoritmo PBKDF2 com SHA256 e milhares de iterações, o que torna inviável a recuperação da senha original a partir do banco. Mesmo com acesso direto ao PostgreSQL, um atacante não conseguiria descobrir as senhas em texto claro dos usuários cadastrados.

A terceira camada são os validadores de senha do Django, configurados em `settings.py`, que rejeitam senhas curtas, comuns ou totalmente numéricas no momento da criação ou alteração. Esses validadores são aplicados tanto no cadastro inicial de um usuário quanto em cada troca de senha, evitando que professores adotem senhas triviais ou semelhantes a atributos pessoais.

A quarta camada é o controle de permissões por papel: as classes `IsAdminRole` e `IsProfessorOrAdmin` são aplicadas declarativamente em cada *endpoint*, restringindo as operações sensíveis — cadastro de professores, edição de perguntas-padrão, painel global — apenas a administradores. A quinta camada é a filtragem do *queryset* no servidor: além da permissão no nível da vista, cada *endpoint* que retorna dados relacionados a disciplinas aplica um filtro adicional, garantindo que professores recebam apenas dados das próprias disciplinas. Essa redundância — permissão mais filtragem — impede que um professor consiga acessar dados de outras disciplinas mesmo manipulando manualmente o identificador na URL.

6.2. Anonimato estrutural dos respondentes

O anonimato dos alunos é garantido de modo estrutural, e não apenas operacional. A entidade `EvaluationResponse`, que registra cada submissão do formulário, não possui qualquer chave estrangeira para `User` — tampouco armazena endereço IP, identificador de sessão ou outro vestígio que permita correlacionar a resposta com um aluno específico. O *endpoint* que recebe a submissão é marcado como público no DRF (`AllowAny`), dispensando a apresentação de *token*. O QR Code direciona o navegador do aluno diretamente para o formulário público, sem qualquer etapa de identificação.

Essa característica é fundamental tanto para a confiança dos respondentes — que precisam saber que suas avaliações não podem ser usadas em retaliação — quanto para o cumprimento dos princípios institucionais de avaliação docente. Vale destacar que mesmo um administrador com acesso direto ao banco de dados não consegue, através do esquema da tabela, deduzir quem deu uma resposta específica. O anonimato é, portanto, uma propriedade do próprio modelo de dados, e não apenas uma promessa operacional do sistema.

6.3. Conformidade com a LGPD

Os dados pessoais tratados pelo sistema são restritos e têm finalidade explícita. São armazenados, apenas para os usuários autenticados (administradores e professores), os atributos nome, sobrenome, e-mail institucional e senha hashada. Conforme o Artigo 6, inciso III da LGPD (Brasil, 2018), o tratamento está limitado ao mínimo necessário para a realização das finalidades do sistema — autenticação e atribuição de disciplinas. Nenhum dado pessoal de aluno é coletado em momento algum.

Quanto aos princípios da finalidade (Art. 6, I) e da adequação (Art. 6, II), o sistema utiliza os dados exclusivamente para o propósito previsto: permitir *login*, atribuir disciplinas a um professor e gerar relatórios consolidados. Quanto ao princípio da segurança (Art. 6, VII), aplicam-se as cinco camadas técnicas descritas na Seção 6.1. Em uma eventual implantação em ambiente de produção, recomenda-se ainda a substituição do CORS aberto por uma lista branca de origens, a obrigatoriedade de HTTPS e a aplicação de limites de requisição no *endpoint* público de submissão do formulário, para mitigar tentativas de *spam* de respostas.

7. Testes e Validação

Este capítulo apresenta a estratégia de testes adotada no projeto. Como o escopo do trabalho não contemplou a produção de testes automatizados, optou-se pela execução de testes manuais, organizados em cenários que cobrem os principais fluxos do sistema. Os testes foram conduzidos pelo próprio autor durante o desenvolvimento, em conjunto com revisões pontuais feitas pelo orientador.

7.1. Estratégia de testes

Os testes foram organizados em torno de seis cenários principais, definidos a partir dos requisitos funcionais discutidos no Capítulo 2: autenticação básica; primeiro *login* com troca obrigatória de senha; cadastro de professor e disciplina, com verificação da geração automática de QR Code e perguntas-padrão; submissão anônima do formulário por um respondente não autenticado; geração do relatório PDF, com verificação visual dos histogramas, percentuais, médias e desvios; e recuperação de senha por e-mail. Cada cenário foi executado de modo iterativo durante o desenvolvimento, e refeito após cada alteração significativa de código. O Quadro 4 sintetiza a matriz de cenários e os requisitos cobertos.

Cenário	Requisitos verificados
Autenticação básica	RF01, RNF03 — JWT emitido apenas com credenciais válidas; mensagem genérica em caso de falha.
Primeiro <i>login</i> com troca de senha	RF01, RF10 — Professor obrigado a definir senha; <i>PrivateRoute</i> bloqueia outras telas.
Cadastro de professor e disciplina	RF02, RF03, RF04, RNF07 — QR Code gerado automaticamente; 24 perguntas-padrão criadas.
Submissão anônima	RF09, RNF05 — <i>Endpoint</i> público; <i>EvaluationResponse</i> sem vínculo com <i>User</i> .
Geração do PDF	RF07, RF08, RNF08 — PDF servido pelo servidor; professor só acessa as próprias disciplinas.
Recuperação de senha	RF10 — <i>Token</i> expirável; resposta genérica que não revela existência do e-mail.

Quadro 4: Matriz de cenários de teste manual.

7.2. Cenários testados e resultados

O cenário de autenticação básica verificou três situações: *login* bem-sucedido de administrador, *login* bem-sucedido de professor (com redirecionamento ao painel adequado) e tentativa de *login* com senha inválida. Em todas as situações o sistema comportou-se conforme esperado, retornando JWT em caso de sucesso e mensagem de erro genérica em caso de falha — sem revelar se o e-mail existe ou não.

O cenário de primeiro *login* verificou que, ao cadastrar um professor com a *flag* de primeiro acesso ativa, este é obrigatoriamente redirecionado para a tela de definição de senha no primeiro acesso, e não consegue navegar para nenhuma outra tela enquanto não concluir a troca. Após a definição da nova senha, a *flag* é desmarcada e o professor passa a operar normalmente. O componente `PrivateRoute` do *front-end* bloqueou efetivamente todas as tentativas de pular essa etapa, mesmo quando o identificador da URL foi manipulado manualmente.

O cenário de cadastro de disciplina verificou três comportamentos automatizados pelo método `save()` do modelo `Course`: a geração do PNG do QR Code no diretório `media/qrcodes/`; a regeneração do QR Code quando o *link* do formulário muda; e a criação das vinte e quatro perguntas-padrão para a nova disciplina. Em todos os casos, os efeitos colaterais foram confirmados diretamente no banco de dados e através da interface administrativa do Django.

O cenário de submissão anônima foi conduzido em duas frentes: a submissão em si, escaneando o QR Code de uma disciplina em um dispositivo móvel; e a confirmação, via consulta direta ao banco, de que a entidade `EvaluationResponse` criada não apresentava qualquer vínculo com a identidade do respondente. Como o cadastro de `Course` usa o *link* público com o identificador da disciplina, o teste também confirmou que múltiplas disciplinas geram QR Codes distintos, cada um apontando para o seu próprio formulário.

O cenário de geração do PDF verificou que o relatório produzido para uma disciplina com respostas reais continha histogramas com porcentagens, médias com desvio padrão, cabeçalho institucional, cálculo amostral e seção de comentários abertos. O layout, a paginação e a numeração dos rodapés foram inspecionados visualmente em múltiplos relatórios. Foi verificada também a regra de acesso: um professor não consegue baixar o relatório de uma disciplina que não lhe pertence — a tentativa, com identificador manipulado, retornou o erro esperado.

Por fim, o cenário de recuperação de senha verificou todo o fluxo: solicitação com e-mail válido (*link* aparece no console do servidor), solicitação com e-mail inexistente (mesma resposta genérica, sem envio de e-mail), uso do *link* para definir nova senha, *login* com

a nova senha e tentativa de reuso do mesmo *link* (rejeitada, pois o *token* foi invalidado após a primeira troca).

7.3. Limitações da estratégia adotada

A principal limitação da estratégia manual é a ausência de cobertura automatizada: cada manutenção futura exige a reexecução manual dos cenários. Em um projeto profissional, recomenda-se a produção de uma suíte de testes automatizados com `pytest` para o *back-end* (testando modelos, serializadores, vistas e permissões) e `Vitest` ou `Jest` para o *front-end* (testando componentes e fluxos com `React Testing Library`). A introdução dessa suíte é listada como trabalho futuro no Capítulo 9.

8. Hospedagem e Disponibilização

Este capítulo descreve a estratégia de hospedagem do sistema. Como o trabalho foi desenvolvido prioritariamente em ambiente local, com o objetivo de demonstração acadêmica, a discussão apresenta tanto o cenário corrente de execução local quanto um caminho viável para a disponibilização em ambiente de produção.

8.1. Execução local

Em ambiente de desenvolvimento, o sistema executa em dois servidores locais. O Django, através de seu servidor de desenvolvimento embutido, responde em `http://127.0.0.1:8000` e expõe a API REST sob o prefixo `/api/`. O Vite, no *front-end*, executa o servidor de desenvolvimento em `http://localhost:5173`, recompilando automaticamente os módulos React modificados. A comunicação entre os dois ocorre por meio das chamadas `Axios` definidas em `src/services/api.js`, com CORS liberado durante o desenvolvimento. Para facilitar a operação, foi escrito o script `iniciar-sistema.bat` na raiz do projeto, que verifica a existência do ambiente virtual Python e do diretório `node_modules`, instala automaticamente as dependências do *front-end* caso ausentes, e abre as duas janelas de servidor em sequência, finalizando com a abertura do navegador no endereço do *front-end*.

8.2. Caminho para produção

Para disponibilizar o sistema em ambiente externo — útil para inspeção pela banca examinadora ou para uma eventual adoção pela FT-UNICAMP — a estratégia mais direta combina dois provedores gratuitos. O *back-end* Django e o banco PostgreSQL podem ser

implantados na plataforma Render, que oferece *tier* gratuito para *web services* Python e instâncias gerenciadas de PostgreSQL com retenção de noventa dias. O *front-end* React, após *build* de produção, pode ser hospedado na plataforma Vercel como aplicação estática, com *deploy* automático a cada novo *commit* no GitHub.

A migração para esse cenário exige seis ajustes de configuração: a inclusão das bibliotecas gunicorn (servidor WSGI), whitenoise (serviço de arquivos estáticos) e dj-database-url (leitura de URL de banco a partir de variável de ambiente) no arquivo requirements.txt; a leitura das variáveis SECRET_KEY, DEBUG, ALLOWED_HOSTS, DATABASE_URL e CORS_ALLOWED_ORIGINS do ambiente em settings.py; a configuração do STATIC_ROOT e do middleware do WhiteNoise; a definição da URL do *back-end* como variável VITE_API_URL no *front-end*; a inclusão de Procfile ou render.yaml indicando o comando de inicialização; e a criação do arquivo runtime.txt com a versão do Python desejada.

Cabe registrar que, em uma eventual adoção definitiva pela FT-UNICAMP, é desejável a hospedagem em servidor próprio da universidade, eliminando a dependência de planos gratuitos externos e simplificando o controle de dados sob a ótica da LGPD.

9. Limitações e Trabalhos Futuros

Este capítulo enumera as limitações do sistema entregue e aponta direções para trabalhos futuros, organizadas por ordem de prioridade percebida.

9.1. Limitações atuais

O sistema entregue cobre integralmente os requisitos definidos no Capítulo 2, mas apresenta limitações que merecem registro. A primeira refere-se à ausência de testes automatizados: a validação foi conduzida exclusivamente por testes manuais, conforme descrito no Capítulo 7. A introdução de uma suíte de testes automatizados — com pytest para o *back-end* e Vitest para o *front-end* — permitiria a detecção mais rápida de regressões em manutenções futuras.

A segunda limitação diz respeito à internacionalização: todos os textos da interface, da documentação e dos relatórios estão em português brasileiro, sem mecanismos de *i18n* configurados. A adoção do sistema fora da FT-UNICAMP exigiria a inclusão de suporte multilíngue. A terceira limitação é a ausência de um aplicativo móvel nativo. Embora a interface seja responsiva, conforme o requisito RNF04, o uso prolongado em *smartphones* e *tablets* se beneficiaria de um aplicativo dedicado, especialmente para professores que desejem consultar relatórios em deslocamento.

A quarta limitação envolve a infraestrutura de e-mail: a recuperação de senha atualmente utiliza o *backend* do console em ambiente de desenvolvimento; em produção, exigirá a configuração de um serviço SMTP real (Gmail, SendGrid ou outro). A quinta refere-se ao escopo das análises estatísticas: o relatório entrega médias, desvios padrão e percentuais por questão, mas não apresenta cruzamentos avançados, como evolução temporal entre semestres ou comparação entre disciplinas. Tais análises são viáveis em trabalhos futuros, já que toda a informação necessária reside no banco de dados.

9.2. Trabalhos futuros

Os principais trabalhos futuros derivados das limitações acima são:

1. a produção de uma suíte de testes automatizados que cubra os fluxos críticos identificados no Capítulo 7, com *pytest* no *back-end* e *Vitest* no *front-end*;
2. a internacionalização da interface, com suporte mínimo a português e inglês, utilizando bibliotecas como *react-i18next*;
3. o desenvolvimento de um aplicativo móvel nativo, reutilizando a API REST já exposta — viável com *React Native* ou *Flutter*;
4. a integração com servidor SMTP institucional para a entrega real dos e-mails de recuperação de senha;
5. a inclusão de uma tela de comparação histórica que apresente a evolução das médias entre semestres por disciplina;
6. a produção de um módulo administrativo de auditoria, que registre as ações sensíveis de administradores e professores;
7. a integração com o sistema acadêmico oficial da FT-UNICAMP para cadastro automático de disciplinas e professores a cada semestre, eliminando a etapa manual de cadastro;
8. a implantação do sistema em servidor próprio da UNICAMP, eliminando a dependência de provedores externos e simplificando o atendimento a requisitos institucionais e legais.

10. Conclusão

Este trabalho apresentou o desenvolvimento de um sistema web para a gestão das avaliações de disciplinas da Faculdade de Tecnologia da UNICAMP. O ponto de partida foi um diagnóstico do processo atual, em que os dados coletados pelos formulários externos eram entregues em formato bruto a uma pessoa responsável pela coordenação do processo — papel hoje exercido pelo próprio orientador deste trabalho — que precisava organizar manualmente os resultados de todas as disciplinas e distribuir um relatório

individual a cada professor. A partir desse diagnóstico, foram derivados dez requisitos funcionais e oito requisitos não funcionais que orientaram o desenvolvimento.

A solução construída cobre o ciclo completo da avaliação: cadastro automatizado de professores, disciplinas e perguntas-padrão; geração automática de QR Codes; coleta anônima de respostas por meio de formulário público; e geração consolidada de relatórios em PDF, com histogramas, percentuais, médias, desvios padrão, comentários abertos e cálculo do tamanho amostral. A arquitetura em três camadas — React, Django REST Framework e PostgreSQL, conectados por API JSON e autenticação JWT — mostrou-se adequada ao escopo do trabalho, permitindo a entrega de todos os requisitos e abrindo caminho para extensões futuras como um aplicativo móvel.

Do ponto de vista prático, espera-se que o sistema, uma vez implantado, reduza significativamente o tempo despendido pelos docentes na produção dos relatórios e padronize o formato dos resultados apresentados em coordenações e processos institucionais. Do ponto de vista de aprendizado, o trabalho integrou conhecimentos adquiridos ao longo do curso de Sistemas de Informação — desenvolvimento web, banco de dados, engenharia de requisitos, modelagem UML, segurança e proteção de dados — em torno de um problema concreto e relevante para a própria faculdade. Os principais trabalhos futuros, conforme detalhado no Capítulo 9, são a produção de testes automatizados, a internacionalização da interface, o desenvolvimento de um aplicativo móvel e a integração com o sistema acadêmico institucional.

O código-fonte completo, esta monografia e o vídeo demonstrativo do sistema estão disponíveis nos repositórios e *links* referenciados no Apêndice A deste documento.

11. Referências

BRASIL. **Lei n. 13.709, de 14 de agosto de 2018. Lei Geral de Proteção de Dados Pessoais (LGPD)**. Disponível em: <http://www.planalto.gov.br/ccivil_03/_ato2015-2018/2018/lei/L13709.htm>.

DJANGO REST FRAMEWORK. **Django REST Framework documentation**. Disponível em: <<https://www.django-rest-framework.org/>>.

DJANGO SOFTWARE FOUNDATION. **Django documentation**. Disponível em: <<https://docs.djangoproject.com/>>.

HUNTER, John D. Matplotlib: A 2D graphics environment. **Computing in Science & Engineering**, v. 9, n. 3, p. 90–95, 2007.

JONES, Michael; BRADLEY, John; SAKIMURA, Nat. **JSON Web Token (JWT)**. Disponível em: <<https://datatracker.ietf.org/doc/html/rfc7519>>.

LUTH, Lincoln. **Python qrcode library**. Disponível em: <<https://pypi.org/project/qrcode/>>.

META PLATFORMS. **React documentation**. Disponível em: <<https://react.dev/>>.

POSTGRES GLOBAL DEVELOPMENT GROUP. **PostgreSQL documentation**. Disponível em: <<https://www.postgresql.org/docs/>>.

REPORTLAB. **ReportLab PDF Toolkit - User Guide**. Disponível em: <<https://www.reportlab.com/docs/reportlab-userguide.pdf>>.

SIMPLE JWT. **django-rest-framework-simplejwt documentation**. Disponível em: <<https://django-rest-framework-simplejwt.readthedocs.io/>>.

YOU, Evan. **Vite documentation**. Disponível em: <<https://vite.dev/>>.

12. Anexos e Apêndices

O código-fonte completo do sistema, a documentação técnica e o vídeo demonstrativo estão disponíveis nos endereços indicados no Apêndice A. Trechos representativos do código, organizados por aplicação, estão reunidos no Apêndice B.

12.1. Apêndice A — Links do Projeto

Repositório do código-fonte. Disponível em: github.com/VictorValentimBergamasco/tcc-avaliacao-disciplinas. O repositório público contém o *back-end* (pasta *backend_tcc*), o *front-end* (pasta *frontend_tcc*), o arquivo *iniciar-sistema.bat* para execução local, o arquivo de documentação técnica em PDF e os arquivos README de instalação.

Vídeo demonstrativo. O vídeo percorre o ciclo completo de operação: *login* do administrador, cadastro de professor e disciplina, geração do QR Code e das perguntas-padrão, primeiro *login* do professor com troca de senha, escaneamento e submissão do formulário por um aluno, e geração final do relatório PDF. Disponível em: <https://youtu.be/Vi92iTVklQ8>.

Documentação técnica. A documentação técnica detalhada complementa esta monografia descrevendo, em nível de código, cada um dos módulos do *back-end* e do *front-end*.

Esse arquivo, denominado `Documentacao_Sistema_TCC.pdf`, está disponível na raiz do repositório indicado acima.

12.2. Apêndice B — Listagens de Código

Este apêndice reúne trechos representativos do código do sistema, organizados por aplicação e arquivo. Os trechos foram selecionados com o objetivo de mostrar concretamente como os comportamentos descritos no Capítulo 5 estão implementados. Listagens completas estão disponíveis no repositório referenciado no Apêndice A.

12.2.1. B.1 Configuração do projeto

O Código 1 apresenta o arquivo `requirements.txt` do back-end, que declara as bibliotecas Python utilizadas. Cada dependência foi escolhida com base na sua robustez, manutenção ativa e integração direta com o Django.

```
Django==5.0.7
django-rest-framework==3.15.2
django-rest-framework-simplejwt==5.3.1
psycogp2-binary==2.9.9
django-cors-headers==4.4.0
qrcode==7.4.2
Pillow==10.4.0
python-dotenv==1.0.1
matplotlib==3.9.2
reportlab==4.2.2
```

Código 1: `backend_tcc/requirements.txt` — dependências Python do back-end.

O Código 2 apresenta o roteamento global do back-end, definido em `core/urls.py`, que encadeia os arquivos `urls.py` de cada aplicação sob o prefixo `/api/`.

```
urlpatterns = [
    path('admin/', admin.site.urls),
    path('api/auth/login/', TokenObtainPairView.as_view()),
    path('api/auth/refresh/', TokenRefreshView.as_view()),
    path('api/users/', include('apps.users.urls')),
    path('api/courses/', include('apps.courses.urls')),
    path('api/evaluations/', include('apps.evaluations.urls')),
    path('api/dashboard/', include('apps.dashboard.urls')),
    path('api/reports/', include('apps.reports.urls')),
]
```

Código 2: `core/urls.py` — roteamento global.

O Código 3 mostra a configuração do PostgreSQL em `core/settings.py`, com leitura das credenciais do arquivo `.env`. O Código 4 apresenta a configuração do JWT e do Django REST Framework no mesmo arquivo.

```
DATABASES = {
    'default': {
        'ENGINE': 'django.db.backends.postgresql',
        'NAME': os.getenv('DB_NAME', 'tcc_avaliacoes'),
        'USER': os.getenv('DB_USER', 'postgres'),
        'PASSWORD': os.getenv('DB_PASSWORD', 'postgres'),
        'HOST': os.getenv('DB_HOST', 'localhost'),
        'PORT': os.getenv('DB_PORT', '5432'),
    }
}
```

Código 3: `core/settings.py` — configuração do banco de dados.

```
SIMPLE_JWT = {
    "ACCESS_TOKEN_LIFETIME": timedelta(hours=4),
    "REFRESH_TOKEN_LIFETIME": timedelta(days=7),
    "AUTH_HEADER_TYPES": ("Bearer",),
}

REST_FRAMEWORK = {
    "DEFAULT_AUTHENTICATION_CLASSES": (
        "rest_framework_simplejwt.authentication.JWTAuthentication",
    ),
    "DEFAULT_PERMISSION_CLASSES": (
        "rest_framework.permissions.IsAuthenticated",
    ),
}
```

Código 4: `core/settings.py` — configuração do JWT e DRF.

12.2.2. B.2 Aplicação users: modelo e permissões

O Código 5 apresenta o modelo User customizado, que substitui o username por `institutional_email` e adiciona os campos `role` e `must_change_password`. O Código 6 mostra as duas classes de permissão usadas em todo o sistema.

```

class User(AbstractUser):
    ROLE_CHOICES = (
        ("admin", "Administrador"),
        ("professor", "Professor"),
    )

    username = None
    email = None

    institutional_email = models.EmailField(unique=True)
    role = models.CharField(
        max_length=20, choices=ROLE_CHOICES, default="professor"
    )
    must_change_password = models.BooleanField(default=False)

    USERNAME_FIELD = "institutional_email"
    REQUIRED_FIELDS = ["first_name", "last_name"]

    objects = UserManager()

```

Código 5: apps/users/models.py — modelo de usuário customizado.

```

class IsAdminRole(permissions.BasePermission):
    def has_permission(self, request, view):
        return (
            request.user.is_authenticated
            and request.user.role == "admin"
        )

class IsProfessorOrAdmin(permissions.BasePermission):
    def has_permission(self, request, view):
        return (
            request.user.is_authenticated
            and request.user.role in ["admin", "professor"]
        )

```

Código 6: apps/users/permissions.py — classes de permissão por papel.

12.2.3. B.3 Aplicação courses: QR Code e padronização automáticos no save()

O Código 7 apresenta a sobrecarga do método `save()` do modelo `Course`, responsável por gerar o QR Code e replicar as perguntas-padrão. O Código 8 mostra o método auxiliar `generate_qr_code`, que constrói a imagem PNG e a atribui ao `ImageField`.

```

def save(self, *args, **kwargs):
    creating = self._state.adding

    should_generate = False
    if self.google_form_link:
        if not self.pk or not self.qr_code_image:
            should_generate = True
        else:
            old = Course.objects.filter(pk=self.pk).first()
            if old and old.google_form_link != self.google_form_link:
                should_generate = True

    if should_generate:
        self.generate_qr_code()

    super().save(*args, **kwargs)

    if creating:
        # Import lazy para evitar import circular evaluations <-> courses
        from apps.evaluations.standard_questions import
sync_standard_questions
sync_standard_questions(self)

```

Código 7: apps/courses/models.py — sobrecarga do método save().

```

def generate_qr_code(self):
    qr = qrcode.make(self.google_form_link)
    buffer = BytesIO()
    qr.save(buffer, format="PNG")
    filename = f"qr_course_{self.code}.png"
    self.qr_code_image.save(filename, File(buffer), save=False)

```

Código 8: apps/courses/models.py — geração do PNG do QR Code.

12.2.4. B.4 Aplicação evaluations: modelos e sincronização

O Código 9 apresenta os modelos da aplicação evaluations, com destaque para a ausência deliberada de chave estrangeira para User no EvaluationResponse. O Código 10 apresenta a função de sincronização do template de perguntas-padrão.

```

class StandardQuestion(models.Model):
    """Pergunta-padrão reutilizada em todas as disciplinas (template)."""
    QUESTION_TYPE_CHOICES = (("scale", "Escala"), ("text", "Texto"))
    text = models.TextField()
    question_type = models.CharField(
        max_length=20, choices=QUESTION_TYPE_CHOICES, default="scale")
    order = models.PositiveIntegerField(unique=True)
    is_active = models.BooleanField(default=True)

class EvaluationQuestion(models.Model):
    course = models.ForeignKey(Course, on_delete=models.CASCADE,
                               related_name="questions")
    text = models.TextField()
    question_type = models.CharField(
        max_length=20, choices=StandardQuestion.QUESTION_TYPE_CHOICES,
        default="scale")
    order = models.PositiveIntegerField(default=1)

class EvaluationResponse(models.Model):
    course = models.ForeignKey(Course, on_delete=models.CASCADE,
                               related_name="responses")
    submitted_at = models.DateTimeField(auto_now_add=True)
    # ANONIMATO: sem ForeignKey para User, sem IP, sem sessão.

class EvaluationAnswer(models.Model):
    response = models.ForeignKey(EvaluationResponse,
                                on_delete=models.CASCADE,
                                related_name="answers")
    question = models.ForeignKey(EvaluationQuestion,
                                on_delete=models.CASCADE,
                                related_name="answers")
    scale_value = models.IntegerField(blank=True, null=True)
    text_value = models.TextField(blank=True, null=True)

```

Código 9: apps/evaluations/models.py — modelos da aplicação.

```

def sync_standard_questions(course, *, overwrite_text=False):
    """Garante que `course` tem as perguntas-padrão do template."""
    from apps.evaluations.models import EvaluationQuestion

    template = _get_template_questions()
    criadas = atualizadas = 0

    for q in template:
        existing = EvaluationQuestion.objects.filter(
            course=course, order=q["order"]
        ).first()
        if existing is None:
            EvaluationQuestion.objects.create(
                course=course, order=q["order"],
                text=q["text"], question_type=q["question_type"],
            )
            criadas += 1
        elif overwrite_text:
            existing.text = q["text"]
            existing.question_type = q["question_type"]
            existing.save(update_fields=["text", "question_type"])
            atualizadas += 1

    return criadas, atualizadas

```

Código 10: apps/evaluations/standard_questions.py — sincronização idempotente.

12.2.5. B.5 Fluxos de primeiro login e recuperação de senha

O Código 11 apresenta a vista que troca a senha do usuário logado. O Código 12 mostra a vista que solicita o e-mail de redefinição. O Código 13 apresenta o componente PrivateRoute do front-end.


```

class ChangePasswordView(APIView):
    permission_classes = [permissions.IsAuthenticated]

    def post(self, request):
        serializer = ChangePasswordSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)

        user = request.user
        current_password = serializer.validated_data.get(
            "current_password", "")
        new_password = serializer.validated_data["new_password"]

        # No primeiro login não exige a senha atual
        if not user.must_change_password:
            if (not current_password
                or not user.check_password(current_password)):
                return Response(
                    {"detail": "Senha atual incorreta."},
                    status=status.HTTP_400_BAD_REQUEST,
                )

        user.set_password(new_password)
        user.must_change_password = False
        user.save(update_fields=["password", "must_change_password"])

        return Response({"detail": "Senha alterada com sucesso."})

```

Código 11: apps/users/views.py — ChangePasswordView.

```

class PasswordResetRequestView(APIView):
    permission_classes = [permissions.AllowAny]

    def post(self, request):
        serializer = PasswordResetRequestSerializer(data=request.data)
        serializer.is_valid(raise_exception=True)
        email = serializer.validated_data["institutional_email"]

        user = User.objects.filter(
            institutional_email__iexact=email, is_active=True
        ).first()

        if user is not None:
            uid = urlsafe_base64_encode(force_bytes(user.pk))
            token = default_token_generator.make_token(user)
            link = (f"{settings.FRONTEND_URL.rstrip('/')}"
                    f"/reset-password/{uid}/{token}")
            send_mail(
                "Recuperação de senha – FT-UNICAMP",
                f"Acesse: {link}",
                settings.DEFAULT_FROM_EMAIL,
                [user.institutional_email],
                fail_silently=False,
            )

        # Resposta GENÉRICA – não revela existência do e-mail
        return Response(
            {"detail": "Se o e-mail estiver cadastrado, enviamos um
link."},
            status=status.HTTP_200_OK,
        )

```

Código 12: apps/users/views.py – PasswordResetRequestView.

```

function PrivateRoute({ children }) {
  if (!localStorage.getItem("token")) {
    return <Navigate to="/" replace />;
  }
  try {
    const user = JSON.parse(localStorage.getItem("user") || "{}");
    if (user.must_change_password) {
      return <Navigate to="/trocar-senha" replace />;
    }
  } catch (e) { /* ignore */ }
  return children;
}

```

Código 13: frontend_tcc/src/App.jsx — componente PrivateRoute.

12.2.6. B.6 Geração do relatório PDF

O Código 14 apresenta a geração do histograma de uma pergunta de escala via Matplotlib. O Código 15 mostra o cálculo da média e do desvio padrão. O Código 16 apresenta o cálculo do tamanho amostral.

```

def create_histogram(values, question_order, output_dir):
    counts = [0, 0, 0, 0, 0, 0]
    for value in values:
        if 0 <= value <= 5:
            counts[value] += 1

    total = sum(counts)
    percentages = [(c / total * 100) if total else 0 for c in counts]

    fig = plt.figure(figsize=(9.5, 5.4), dpi=120)
    ax = fig.add_subplot(111)
    bars = ax.bar(range(6), counts, alpha=0.8)

    for i, bar in enumerate(bars):
        ax.text(bar.get_x() + bar.get_width()/2,
                bar.get_height() + 0.5,
                f"{percentages[i]:.1f} %",
                ha="center", fontweight="bold")

    ax.set_xticklabels(SCALE_LABELS)
    ax.set_ylabel("Número de Alunos")

    path = Path(output_dir) / f"q{question_order}.png"
    fig.savefig(path, bbox_inches="tight")
    plt.close(fig)
    return path

```

Código 14: apps/reports/services.py — geração do histograma.

```

def calculate_mean_sd(values):
    # Ignora "Não sei avaliar" (valor 0) no cálculo estatístico
    valid = [v for v in values if v > 0]
    if not valid:
        return None, None

    mean = sum(valid) / len(valid)
    variance = sum((x - mean) ** 2 for x in valid) / len(valid)
    return mean, math.sqrt(variance)

```

Código 15: apps/reports/services.py — média e desvio padrão.

```

def tamanho_amostrai(matriculados, tipo="professor"):
    try:
        n_pop = int(matriculados)
    except (TypeError, ValueError):
        n_pop = 0

    if n_pop <= 0:
        return 0

    z = 1.96 # 95% de confiança
    p = 0.5 # variância máxima
    e = 0.15 if tipo == "professor" else 0.05

    aa = (z * z * p * (1 - p)) / (e * e)
    return aa / (1 + (aa / n_pop))

```

Código 16: apps/reports/services.py — cálculo do tamanho amostral.

12.2.7. B.7 Filtragem por papel e dashboard

O Código 17 mostra a filtragem do queryset por papel. O Código 18 apresenta o endpoint que alimenta o gráfico de médias por pergunta.

```

class CourseListCreateView(generics.ListCreateAPIView):
    serializer_class = CourseSerializer
    permission_classes = [IsProfessorOrAdmin]

    def get_queryset(self):
        user = self.request.user
        if user.role == "admin":
            return Course.objects.select_related("professor").all()
        return Course.objects.select_related("professor").filter(
            professor=user
        )

    def get_permissions(self):
        if self.request.method == "POST":
            return [IsAdminRole()]
        return super().get_permissions()

```

Código 17: apps/courses/views.py — filtragem por papel.

```

class CourseDashboard(APIView):
    permission_classes = [IsProfessorOrAdmin]

    def get(self, request, course_id):
        course = get_object_or_404(Course, id=course_id)

        user = request.user
        if (user.role == "professor"
            and course.professor_id != user.id):
            raise PermissionDenied(
                "Você não tem permissão para esta disciplina."
            )

        questions = EvaluationQuestion.objects.filter(
            course=course).order_by("order")

        result = []
        for q in questions:
            answers = EvaluationAnswer.objects.filter(
                question=q, scale_value__isnull=False)
            avg = answers.aggregate(avg=Avg("scale_value"))["avg"]
            result.append({
                "question": q.text,
                "order": q.order,
                "average": round(avg, 2) if avg else 0,
                "total_answers": answers.count(),
            })

        return Response({"course": course.name, "questions": result})

```

Código 18: apps/dashboard/views.py — endpoint do dashboard.

12.2.8. B.8 Front-end: cliente HTTP, Layout e Login

O Código 19 apresenta o módulo central de comunicação com a API. O Código 20 mostra o componente Layout que resolve automaticamente o papel do usuário. O Código 21 mostra o handler de login com redirecionamento condicional.

```

import axios from "axios";

const api = axios.create({
  baseURL: "http://127.0.0.1:8000/api/",
});

api.interceptors.request.use((config) => {
  const token = localStorage.getItem("token");
  if (token) {
    config.headers.Authorization = `Bearer ${token}`;
  }
  return config;
});

export function getMediaUrl(pathOrUrl) {
  if (!pathOrUrl) return "";
  if (pathOrUrl.startsWith("http")) return pathOrUrl;
  return `http://127.0.0.1:8000${pathOrUrl}`;
}

export default api;

```

Código 19: frontend_tcc/src/services/api.js — cliente HTTP centralizado.

```

import Sidebar from "./Sidebar";

function resolveRole(propRole) {
  if (propRole) return propRole;
  try {
    const user = JSON.parse(localStorage.getItem("user") || "{}");
    if (user.role === "professor") return "Professor";
    if (user.role === "admin") return "Admin";
  } catch (e) { /* ignore */ }
  return "Admin";
}

export default function Layout({ children, role, userName = "" }) {
  const finalRole = resolveRole(role);
  let displayName = userName;
  if (!displayName) {
    try {
      const user = JSON.parse(localStorage.getItem("user") ||
"{}");
      displayName = user.full_name || user.institutional_email ||
"";
    } catch (e) { /* ignore */ }
  }
  return (
    <div className="app-shell">
      <Sidebar role={finalRole} />
      <main className="main-content">
        <header className="topbar">
          {displayName || "Sistema de Avaliação"}
        </header>
        {children}
      </main>
    </div>
  );
}

```

Código 20: frontend_tcc/src/components/Layout.jsx — componente Layout.


```

async function handleLogin(e) {
  e.preventDefault();
  setError("");
  try {
    const r = await api.post("auth/login/", {
      institutional_email: email, password,
    });
    localStorage.setItem("token", r.data.access);

    const me = await api.get("users/me/");
    localStorage.setItem("user", JSON.stringify(me.data));

    if (me.data.must_change_password) {
      window.location.href = "/trocar-senha";
      return;
    }

    window.location.href = me.data.role === "professor"
      ? "/professor" : "/admin";
  } catch {
    setError("Email ou senha inválida");
  }
}

```

Código 21: frontend_tcc/src/pages/Login.jsx — handler de login.